

Trabalho Prático: Implementação de Testes de Software

Professor: Marcelo Arantes de Oliveira

Curso: Engenharia de Software

Modalidade: Individual ou em duplas

Data de Entrega: 05 de Outubro (até as 23h59)

Local de Entrega: Plataforma NEAD

Valor: 3,0 pontos

1. Objetivo

O objetivo deste trabalho prático é aplicar os conceitos de Engenharia de Software focados em Qualidade, construindo uma suíte abrangente de testes automatizados (Unitários, de Componente e de Integração) para uma aplicação Python existente.

2. Repositório Base

O código-fonte da aplicação de Controle de Gastos Familiares está disponível no repositório abaixo. Vocês devem realizar um *fork* ou clonar o repositório para iniciar o desenvolvimento da suíte de testes:

 **Repositório:** <https://github.com/marceloarantes19/TesteDeSoftware---2>

3. Escopo do Trabalho

O trabalho consiste em desenvolver testes para as seguintes classes de domínio da aplicação:

- Categoria
- FormaPagamento (Tratada no código como Forma de Pagamento)
- Pessoa
- Movimento

A dupla deve utilizar um framework de testes da linguagem Python (recomenda-se o **pytest** ou o módulo nativo **unittest**). Os testes deverão ser divididos nas seguintes categorias:

A. Testes Unitários (1,0 ponto)

Os testes unitários devem validar o comportamento das classes de forma estritamente isolada.

- **O que testar:**
 - Validações de entrada de dados (ex: verificar se os *setters* das classes Pessoa e Movimento levantam erro ao receber uma data no formato incorreto).
 - Geração correta de UUIDs quando um objeto é instanciado sem um ID prévio.
 - Funcionamento correto dos métodos de serialização e desserialização (*to_dict* e *from_dict*).
- **Restrição:** As dependências externas ou interações com o sistema de arquivos devem ser isoladas (usando *Mocks*, se necessário).

B. Testes de Componente (1,0 ponto)

Os testes de componente devem avaliar o funcionamento das classes de domínio quando acopladas ao seu mecanismo de persistência ou a módulos adjacentes imediatos.

- **O que testar:**
 - Interação estrutural entre as classes base (ex: Categoria, Pessoa) e a classe genérica RepositorioCSV.
 - Garantir que a gravação, leitura, atualização e remoção (CRUD) de uma entidade no arquivo CSV temporário funcione conforme esperado para aquele módulo específico, simulando o ambiente real de execução do componente.

C. Testes de Integração (1,0 ponto)

Os testes de integração devem validar o fluxo de dados cruzando múltiplas camadas ou múltiplas classes trabalhando em conjunto.

- **O que testar:**
 - A criação de um cenário completo: Instanciar uma Pessoa associada a uma FuncaoFamiliar e, em seguida, registrar um Movimento financeiro válido que integre essa Pessoa, uma Categoria e uma FormaPagamento.
 - Validar se as chaves de relacionamento (responsavel_id, categoria_id, etc.) estão sendo referenciadas e recuperadas corretamente em um fluxo completo da aplicação.

4. O que deve ser entregue no NEAD

Um (apenas um) integrante da dupla deverá submeter na área de tarefas do NEAD um **arquivo compactado (.zip)** contendo:

1. A pasta do projeto atualizada com um subdiretório tests/ contendo todos os arquivos de teste desenvolvidos (ex: test_pessoa.py, test_movimento.py, etc.).
2. Um arquivo requirements.txt atualizado, caso tenham adicionado bibliotecas extras (como pytest ou pytest-mock).
3. Um breve arquivo LEIA-ME.md (ou README.md) explicando os comandos necessários para instalar as dependências e rodar a suíte completa de testes.

5. Critérios de Avaliação

- **Cobertura e Abrangência:** Os testes cobrem cenários de sucesso (caminho feliz) e cenários de falha/exceções (ex: tentar criar um movimento com data inválida)?
- **Isolamento:** Os testes unitários estão de fato isolados, sem gravar arquivos reais no disco do computador durante a execução?
- **Boas Práticas:** Uso correto de *fixtures* (como setUp e tearDown ou @pytest.fixture), organização do código e nomenclatura clara para os métodos de teste.
- **Execução:** A suíte roda com sucesso (passando ou apontando falhas reais do código base) utilizando o comando padrão de testes.

Dica: Lembrem-se de que um bom teste não é aquele que apenas passa, mas aquele que garante o comportamento correto e acusa o erro quando a regra de negócio for violada.
Bom trabalho!